

Documentation technique — Vite & Gourmand

1. Réflexions initiales technologiques et justification des choix

3.1 Langages et architecture

PHP 8.3 (licence PHP, open source et gratuit). PHP est le langage imposé par le cadre du projet et le cœur du référentiel DWWM côté back-end. Le choix d'un développement **sans framework** est volontaire et pédagogique : il démontre la compréhension des mécanismes que les frameworks automatisent, notamment le routage, l'autoloading, l'injection de la connexion à la base et la séparation des responsabilités. La version 8.3 apporte le typage strict, les propriétés typées et des performances améliorées, utilisés dans l'ensemble du code.

Architecture MVC avec routeur personnalisé. L'application sépare strictement les modèles (classes `Table` chargées des requêtes), les vues (templates PHP) et les contrôleurs (logique applicative). Un fichier `routes.php` associe chaque URL à un contrôleur pleinement qualifié par son namespace. Les routes sont préfixées par espace (`employe-menus` , `admin-dashboard` , etc.) afin d'éviter les collisions de noms entre espaces, et chaque espace dispose de ses propres contrôleurs plutôt que d'un contrôleur partagé rempli de conditions sur le rôle : le code reste lisible, testable et conforme au principe de responsabilité unique.

Organisation de la couche Vue : layouts et partials. La couche Vue est elle-même structurée en deux niveaux. Les *layouts* (gabarits de page), un par espace — `layout-public.php` , `layout-user.php` , `layout-employe.php` et `layout-admin.php` — définissent le squelette HTML commun de chaque espace : en-tête de document, feuilles de style, scripts et navigation propre au rôle. Chaque vue rendue par un contrôleur est injectée dans le layout de son espace, ce qui évite de dupliquer la structure HTML sur l'ensemble des pages et garantit une présentation cohérente par rôle. Les *partials* sont des fragments réutilisables inclus dans les layouts et les vues : en-tête, pied de page, barres latérales spécifiques à chaque rôle (`sidebar-user` , `sidebar-employe` , `sidebar-admin`) ainsi que l'ensemble des popups d'action (ajout, modification et suppression de plats et de menus, changement de statut de commande, gestion des comptes employés), chacune isolée dans son propre fichier sous `View/partials/popups/` . Cette organisation applique le principe DRY (*Don't Repeat Yourself*) à la couche présentation et prolonge la logique de séparation par espace déjà appliquée aux contrôleurs et au middleware.

JavaScript (vanilla). Le JavaScript natif est utilisé sans bibliothèque front lourde : manipulation du DOM pour les popups dynamiques de détail de menu, et `fetch()` pour le filtrage AJAX des menus sans rechargement de page. Ce choix évite une dépendance supplémentaire pour des besoins que le langage natif couvre parfaitement, et démontre la maîtrise des fondamentaux exigée par le référentiel.

HTML5 / CSS3. Structure sémantique et mise en page responsive afin de garantir la consultation sur mobile, tablette et ordinateur, conformément au cahier des charges.

3.2 Bases de données

MySQL (licence GPL, édition communautaire gratuite), via PDO. Les données métier (utilisateurs, rôles, menus, plats, thèmes, régimes, commandes, historique de statuts, photos) sont fortement structurées et liées entre elles : un modèle relationnel s'impose pour garantir l'intégrité référentielle (clés étrangères), les jointures et les transactions. L'accès se fait exclusivement via PDO et des requêtes préparées, ce qui protège des injections SQL et permettrait de changer de SGBD avec un impact minimal. Les photos des menus sont stockées en BLOB dans une table dédiée `menu_photos`, ce qui simplifie la sauvegarde et le déploiement (aucun dossier d'uploads à synchroniser) pour une volumétrie adaptée au projet.

MongoDB (Community Server, licence SSPL, gratuit). Le cahier des charges impose l'utilisation d'une base NoSQL. MongoDB stocke les données statistiques du tableau de bord administrateur : ces données, orientées documents et sans besoin de jointures, se prêtent naturellement à un stockage souple dont le schéma peut évoluer sans migration. Le driver PHP officiel (extension `mongodb` + bibliothèque `mongodb/mongodb` via Composer) assure la communication avec le serveur.

3.3 Outils et bibliothèques

Composer (licence MIT). Gestionnaire de dépendances de l'écosystème PHP. Il fournit l'autoloading PSR-4 (namespace `App` mappé sur `src/App/`), ce qui élimine les `require` manuels, et gère l'installation et la mise à jour des bibliothèques tierces (`composer audit` permet en outre de vérifier les vulnérabilités connues des dépendances).

PHPMailer (licence LGPL 2.1). Bibliothèque de référence pour l'envoi d'emails en PHP, utilisée pour les emails transactionnels (confirmation de commande, notifications). Elle gère le SMTP authentifié, ce que la fonction native `mail()` ne couvre pas correctement, et bénéficie d'une maintenance active — un critère important au regard de son historique de sécurité (voir la veille technologique, section 6).

Git et GitHub (Git : GPLv2 ; GitHub : gratuit pour les dépôts). Versioning de l'intégralité du projet : historique des modifications, possibilité de revenir en arrière, sauvegarde distante et support de la livraison du code, GitHub étant le canal de rendu demandé. Le fichier `.gitignore` exclut les fichiers sensibles (configuration d'environnement) et les dépendances (`vendor/`).

MAMP (version de base gratuite). Environnement de développement local fournissant Apache, MySQL et PHP préconfigurés sur macOS, avec sélection de la version de PHP. Le détail de la mise en place de l'environnement et sa justification figurent dans le `README.md` du dépôt (voir livrable dédié).

Railway (plateforme cloud, offre de démarrage). Plateforme de déploiement retenue pour la mise en ligne, déjà éprouvée sur un précédent projet personnel (EasyOrganis) : dé-

ploiement depuis GitHub, gestion des variables d'environnement et provisionnement des bases de données.

3.4 Synthèse des choix écartés

Symfony et Laravel ont été écartés car le projet vise précisément à démontrer la maîtrise des mécanismes sous-jacents sans framework. Une bibliothèque front comme React aurait été surdimensionnée pour des interactions ciblées (popups, filtrage AJAX) réalisables en JavaScript natif. Enfin, le stockage des photos sur le système de fichiers a été écarté au profit du BLOB pour simplifier la portabilité de l'application dans le cadre du projet.

3.5 Déploiement en production

Railway a été retenu comme plateforme d'hébergement pour sa gestion intégrée de plusieurs services (application, base MySQL, base MongoDB) au sein d'un même projet, son déploiement continu depuis GitHub, et sa gestion centralisée des variables d'environnement.

Choix du système de build : Railpack (FrankenPHP). Une première approche par conteneurisation Docker manuelle a été explorée, avec un `Dockerfile` complet (image `php:8.3-apache`, extensions PDO MySQL et MongoDB compilées via `pecl`, configuration Apache dédiée). Cette approche a révélé un conflit de configuration Apache (chargement simultané de deux modules MPM, `mpm_prefork` et `mpm_event`), provoquant l'échec systématique du démarrage du conteneur malgré plusieurs correctifs. Le choix a donc été fait d'abandonner la gestion manuelle du serveur web au profit de **Railpack**, le système de build natif de Railway, qui détecte automatiquement le langage du projet et provisionne un environnement **FrankenPHP** (serveur d'application PHP moderne basé sur Caddy) sans configuration Apache à maintenir soi-même. Cette décision illustre un principe de développement pragmatique : préférer une solution native bien supportée par la plateforme cible plutôt que de maintenir une configuration bas niveau sujette à des conflits difficiles à diagnostiquer.

Configuration du serveur web via `Caddyfile`. FrankenPHP s'appuie sur Caddy comme serveur web. Un fichier `Caddyfile` à la racine du projet définit la configuration minimale nécessaire :

```
{
    admin off
}

:{$PORT} {
    root * public
    encode gzip
    php_server
    file_server
}
```

Ce fichier indique que la racine web est le dossier `public/` (où se trouve `index.php`), que les requêtes PHP sont traitées par `php_server`, et que les fichiers statiques (CSS, JavaScript,

images) sont servis directement par `file_server`. Le port d'écoute est fourni dynamiquement par Railway via la variable `$PORT`, différente à chaque déploiement.

Extension MongoDB et build personnalisé. L'extension PHP `mongodb`, nécessaire au tableau de bord statistique, n'étant pas installée par défaut dans l'environnement Railpack, une commande de build personnalisée a été configurée dans les paramètres Railway afin d'ignorer la vérification de plateforme au moment de l'installation des dépendances Composer, puis d'installer l'extension `pdo_mysql` nécessaire à la connexion MySQL :

```
composer install --ignore-platform-req=ext-mongodb --optimize-autoloader --no-dev --no-interaction && docker-php-ext-install pdo_mysql
```

Bases de données provisionnées séparément. MySQL et MongoDB sont chacune déployées comme un service Railway indépendant, avec leurs propres identifiants de connexion générés automatiquement. L'application communique avec elles via le réseau privé Railway (`*.railway.internal`), plus rapide et non facturé au trafic, contrairement au réseau public utilisé ponctuellement pour l'import initial des données via un client `mysql` en ligne de commande depuis le poste de développement.

Gestion des chemins entre environnement local et production. Le développement local sous MAMP plaçait l'application dans un sous-dossier de `htdocs/`, ce qui produisait des URLs locales de la forme `localhost:8888/vitegourmand/public/...`. Plusieurs chemins absolus dans les vues, les scripts JavaScript et les redirections serveur avaient été écrits en dur avec ce préfixe. En production, `public/` étant directement la racine du serveur web, ces préfixes ont dû être supprimés dans l'ensemble des templates, partiels, pages et contrôleurs concernés, à l'exception du lien de réinitialisation de mot de passe envoyé par email, qui nécessite une URL absolue complète : celle-ci est désormais construite à partir d'une variable d'environnement `APP_URL`, différente en local et en production, plutôt que codée en dur. Cette correction illustre l'importance de ne jamais coupler la structure du code à un chemin de déploiement particulier.

Lecture des variables d'environnement. La configuration de connexion aux bases de données (`config/settings.php`) lit les variables via `$_ENV[]` avec, en complément, un `repli` sur `getenv()`. Cette double lecture est nécessaire car la directive PHP `variables_order`, qui détermine si les variables d'environnement système alimentent `$_ENV`, n'est pas configurée de façon identique dans tous les environnements d'exécution ; `getenv()` reste fiable indépendamment de ce réglage.

2. Configuration de l'environnement de travail

Comment j'ai travaillé sur ce projet

Je développe en local avec **MAMP**, qui installe en une fois Apache, MySQL et PHP sur macOS, avec phpMyAdmin pour gérer la base sans passer par la ligne de commande à chaque fois, et la possibilité de choisir précisément la version de PHP (8.3, la même qu'en production). Pour un projet PHP classique développé seul, c'est plus rapide à mettre en place qu'une solu-

tion conteneurisée type Docker, tout en restant proche d'un vrai environnement serveur. Apache tourne sur le port **8888**, MySQL sur le port **8889**.

J'écris le code dans **Visual Studio Code**, avec quelques extensions utiles au quotidien : PHP Intelephense pour l'autocomplétion, GitLens pour naviguer dans l'historique Git, et le terminal intégré pour ne jamais changer de fenêtre entre écrire, tester et commiter.

Composer structure le projet dès le départ : l'autoload PSR-4 (`App\` → `src/App/`) organise les classes par namespace et évite d'écrire des `require` à la main partout, ce qui devient vite indispensable avec autant de contrôleurs et de modèles répartis par espace (public, client, employé, admin).

Git et GitHub sont utilisés depuis le premier jour, avec des commits réguliers et un dépôt distant qui sert à la fois de sauvegarde et de canal de rendu du projet. Le fichier `.gitignore` exclut le dossier `vendor/` (réinstallable en une commande) et la configuration contenant les mots de passe.

MongoDB m'a demandé un peu plus de travail en local : le driver PHP (l'extension `mongodb`) n'est pas installé par défaut, j'ai dû le compiler moi-même avec les outils fournis par PHP 8.3 sur MAMP, en parallèle d'un serveur MongoDB Community qui tourne à côté de MySQL. Ce travail permet au tableau de bord statistique de fonctionner en local exactement comme en production.

Le déploiement sur Railway

C'est la partie qui m'a pris le plus de temps à comprendre. Au départ, j'avais prévu de conteneuriser l'application avec **Docker** : une image basée sur `php:8.3-apache` , avec les extensions PDO MySQL et MongoDB installées manuellement. Mais Apache refusait de démarrer à cause d'un conflit entre deux modules internes (`mpm_prefork` et `mpm_event` chargés en même temps), et malgré plusieurs tentatives de correction, le problème persistait.

J'ai donc changé d'approche et laissé Railway gérer ça lui-même via **Railpack**, son système de build natif : il détecte automatiquement que le projet est en PHP et prépare un environnement **FrankenPHP** (un serveur d'application PHP moderne, basé sur Caddy) sans que j'aie besoin de maintenir une configuration Apache à la main. C'est un bon exemple de choix pragmatique : plutôt que de s'acharner sur une configuration bas niveau compliquée à déboguer, j'ai préféré une solution native déjà bien intégrée à la plateforme.

Un fichier `Caddyfile` à la racine du projet indique où se trouve la racine web (`public/` , là où est `index.php`) et active à la fois le traitement du PHP et le service des fichiers statiques (CSS, JS, images). Une commande de build personnalisée installe en plus l'extension `pdo_mysql` , nécessaire à la connexion MySQL et absente par défaut de l'environnement Railpack.

Déploiement — en résumé

- **Plateforme** : Railway, build via Railpack, environnement FrankenPHP
- **Bases de données** : un service MySQL et un service MongoDB séparés, chacun avec ses propres identifiants, connectés à l'application via le réseau privé Railway

- **Serveur web** : configuré par le `Caddyfile` à la racine (racine web `public/`, PHP et fichiers statiques servis)
- **Variables d'environnement** : gérées depuis l'interface Railway (connexion base de données, MongoDB, SMTP, et `APP_URL` pour construire les liens envoyés par email)
- **Branches** : le déploiement se déclenche automatiquement à chaque push sur `main` ; le travail au jour le jour se fait sur `develop` et des branches de fonctionnalité qui en découlent

3. Modèle conceptuel de données

Le modèle conceptuel de données est fourni en document séparé : `diagramme-classes.pdf`, présent dans ce même dossier `docs/`.

4. Diagramme d'utilisation et diagramme de séquence

Ces deux diagrammes sont fournis en documents séparés : `diagramme-cas-utilisation.pdf` et `diagramme-sequence.pdf`, présents dans ce même dossier `docs/`.

5. Mécanismes de sécurité

La sécurité a été traitée à trois niveaux complémentaires : les formulaires, le back-end et le front-end. La démarche s'appuie sur les recommandations de l'OWASP (Top 10) présentées dans la veille technologique (section 6).

5.1 Sécurisation des formulaires

- **Double validation, côté client et côté serveur.** Les formulaires utilisent les attributs HTML5 (`required`, `type="email"`, `minlength`, etc.) pour un premier contrôle immédiat côté navigateur, mais chaque donnée est systématiquement revalidée côté serveur (contrôles de présence, de format via `filter_var()`, de longueur et de cohérence métier), car la validation client peut être contournée.
- **Assainissement des entrées.** Les données reçues sont nettoyées (`trim()`, contrôle des types) avant tout traitement ; aucune donnée utilisateur n'est utilisée telle quelle dans une requête ou une vue.
- **Protection contre le XSS à l'affichage.** Toute donnée d'origine utilisateur affichée dans une vue est échappée avec `htmlspecialchars()`, empêchant l'injection de balises ou de scripts dans les pages (menus, commandes, profils).
- **Messages d'erreur génériques à l'authentification.** En cas d'échec de connexion, le message ne précise pas si c'est l'email ou le mot de passe qui est erroné, afin de ne pas faciliter l'énumération de comptes.
- **Contrôle des fichiers envoyés.** Les photos de menus sont contrôlées avant enregistrement en BLOB : vérification du type et de la taille, afin d'empêcher le dépôt de fichiers arbitraires.

5.2 Sécurisation back-end

- **Requêtes préparées PDO généralisées.** Toutes les requêtes SQL passent par des requêtes préparées avec paramètres liés, dans l'ensemble des classes `Table`. C'est la protection de référence contre l'injection SQL (catégorie A03 de l'OWASP Top 10).
- **Hachage des mots de passe avec bcrypt.** Les mots de passe sont hachés avec `password_hash()` (algorithme bcrypt, salage automatique) et vérifiés avec `password_verify()`. Aucun mot de passe n'est stocké ni journalisé en clair.
- **Contrôle d'accès par rôle via un middleware.** La classe `AuthMiddleware` expose quatre niveaux de contrôle (`check`, `checkClient`, `checkEmploye`, `checkAdmin`) appliqués aux routes protégées. La vérification s'appuie sur le `role_id` stocké en session côté serveur : un client ne peut pas atteindre une URL de l'espace employé ou admin, même en la saisissant directement. Ce mécanisme répond à la catégorie « Broken Access Control » (A01), première du classement OWASP.
- **Gestion durcie des sessions.** Expiration automatique après 20 minutes d'inactivité (1200 secondes) avec invalidation de la session et redirection vers la connexion ; destruction complète de la session à la déconnexion.
- **Logique métier vérifiée côté serveur.** Les règles sensibles, comme les conditions d'annulation d'une commande selon le rôle, sont appliquées côté serveur et jamais déléguées au front.
- **Protection des secrets.** Les identifiants de connexion aux bases de données et au serveur SMTP sont placés dans un fichier de configuration d'environnement exclu du versioning via `.gitignore` ; un fichier d'exemple sans valeurs sensibles est fourni à la place. En complément, une procédure de rotation des identifiants est appliquée en cas de doute sur leur exposition.
- **Maîtrise des erreurs.** L'affichage des erreurs PHP est réservé à l'environnement de développement ; en production, les erreurs ne sont pas exposées à l'utilisateur afin de ne divulguer ni chemins, ni requêtes, ni structure interne.

5.3 Sécurisation front-end

- **Aucune donnée sensible côté client.** Aucun mot de passe, identifiant technique ou information de session n'est stocké dans le code JavaScript, le `localStorage` ou les cookies accessibles aux scripts ; l'état d'authentification vit exclusivement dans la session côté serveur.
- **Insertion sûre dans le DOM.** Les données injectées dynamiquement (résultats du filtrage AJAX, contenus des popups) sont insérées en privilégiant `textContent` et des constructions maîtrisées plutôt que la concaténation directe de HTML, afin de limiter les vecteurs XSS côté client.
- **Endpoints AJAX protégés.** Les routes appelées en `fetch()` passent par le même routeur et le même middleware que les pages classiques : le filtrage AJAX ne constitue pas une porte dérobée contournant le contrôle d'accès.

- **HTTPS en production.** Le déploiement s'effectue derrière HTTPS, chiffrant les échanges (identifiants de connexion, données personnelles) entre le navigateur et le serveur.
- **Adaptation mobile (responsive design).** L'interface est adaptée à trois plages d'affichage (desktop, tablette, mobile) via des media queries CSS. Sur mobile, la navigation latérale (sidebar) est remplacée par un menu accessible via un bouton dédié, les tableaux (commandes, plats, menus) sont transformés en cartes empilées verticalement plutôt que de forcer un défilement horizontal, et les popups occupent une largeur adaptée à l'écran avec défilement interne. Cette adaptation garantit que les fonctionnalités sensibles (annulation de commande, changement de statut) restent utilisables sur tout type d'appareil, condition implicite du livrable de charte graphique qui exige des maquettes mobiles.

5.4 Traçabilité et gestion du prêt de matériel

Le cahier des charges prévoit qu'une commande puisse s'accompagner d'un prêt de matériel, dont la restitution est encadrée par un délai contractuel. Ce mécanisme combine traçabilité en base de données et information automatique du client.

- **Modélisation en base.** La table `commande` comporte deux colonnes booléennes : `pret_materiel`, qui indique si la commande concernée inclut la mise à disposition de matériel, et `restitution_materiel`, qui trace si ce matériel a été effectivement rendu.
- **Extension du cycle de statuts.** Le cycle standard d'une commande (`en attente` → `en préparation` → `prête` → `livrée`) est complété, uniquement lorsque `pret_materiel` vaut vrai, par un statut intermédiaire `en attente du retour de matériel` entre `livrée` et `terminée`. Une commande sans prêt de matériel passe directement de `livrée` à `terminée`. Les transitions autorisées sont contrôlées côté serveur dans le contrôleur de gestion des commandes, empêchant qu'un statut soit atteint dans le désordre.
- **Notification automatique par email.** Le passage au statut `en attente du retour de matériel` déclenche l'envoi automatique d'un email (PHPMailer) informant le client qu'il dispose de **10 jours ouvrés** pour restituer le matériel, au-delà desquels des **frais de 600 €** sont appliqués conformément aux conditions générales de vente, et qu'il doit reprendre contact avec l'entreprise pour organiser cette restitution — aucune action en libre-service n'est proposée au client pour cette étape, la restitution étant physiquement constatée par un employé.
- **Traçabilité temporelle.** Chaque changement de statut, y compris ce statut intermédiaire, est enregistré avec horodatage dans la table `historique_statut`, permettant de retrouver à quelle date le délai de 10 jours a commencé à courir.
- **Clôture du prêt.** Le passage manuel du statut `en attente du retour de matériel` à `terminée`, effectué par un employé après constat physique du retour, met à jour `restitution_materiel` à vrai, clôturant la traçabilité de ce prêt.

5.5 Axes d'amélioration identifiés

Dans une démarche d'amélioration continue, plusieurs renforcements sont identifiés pour la suite : ajout de jetons anti-CSRF sur les formulaires sensibles, limitation du nombre de tentatives de connexion (protection contre la force brute), régénération systématique de l'identifiant de session après connexion et mise en place d'en-têtes de sécurité HTTP (Content-Security-Policy, X-Frame-Options).

6. Veille technologique sur les vulnérabilités de sécurité

6.1 Définition et objectifs de ma veille

La veille technologique consiste à se tenir informé de manière régulière et organisée des évolutions de son domaine. Dans le cadre de ce projet, je l'ai orientée vers la sécurité, avec trois objectifs : identifier les vulnérabilités connues touchant les technologies de mon stack (PHP, MySQL, MongoDB, bibliothèques Composer), comprendre les techniques d'attaque les plus courantes pour mieux m'en protéger, et confronter mes choix techniques aux bonnes pratiques recommandées par les organismes de référence.

6.2 Méthodologie et sources

Pour ce projet, j'ai concentré ma veille sur les ressources suivantes :

- **OWASP Top 10** : le référentiel des dix risques de sécurité les plus critiques pour les applications web, que j'ai utilisé comme grille de lecture pour analyser mon propre code ;
- **La base CVE (Common Vulnerabilities and Exposures)** : pour rechercher les vulnérabilités documentées touchant précisément les technologies que j'utilise (PHP, PHPMailer) ;
- **composer audit** : commande intégrée à Composer qui vérifie si les dépendances installées dans le projet correspondent à des versions présentant des vulnérabilités connues et publiées.

Cette démarche n'est pas un dispositif permanent que je maintiens au quotidien, mais une recherche ciblée menée pendant le développement de Vite & Gourmand, pour m'assurer que les choix techniques du projet (bibliothèques utilisées, configuration serveur) ne reproduisent pas des failles déjà identifiées ailleurs. C'est une pratique que je compte approfondir et rendre plus régulière dans le cadre de mon alternance.

6.3 Sujet d'étude : l'OWASP Top 10 appliqué au projet

J'ai confronté chaque catégorie de l'OWASP Top 10 (édition 2021) à Vite & Gourmand, pour identifier les risques les plus pertinents compte tenu de l'architecture du projet. Trois catégories se sont révélées particulièrement structurantes.

A01 - Broken Access Control, première du classement : un contrôle d'accès défaillant permet à un utilisateur d'atteindre des ressources qui ne lui sont pas destinées. C'est ce risque qui a motivé la mise en place du middleware à quatre niveaux (`check`, `checkClient`,

`checkEmploye`, `checkAdmin`) et la vérification systématique du rôle en session sur chaque route protégée.

A03 - Injection : les injections SQL restent l'une des attaques les plus répandues contre les applications PHP. C'est ce risque qui a justifié la généralisation des requêtes préparées PDO dans l'ensemble des classes `Table` du projet, plutôt qu'une concaténation directe des requêtes SQL.

A07 - Identification and Authentication Failures : cette catégorie recouvre les mots de passe mal stockés et les sessions mal gérées. Elle a confirmé la pertinence du choix de bcrypt via `password_hash()` et de l'expiration automatique de session après inactivité, déjà prévus dans le cahier des charges du projet.

6.4 Deux vulnérabilités documentées, en lien avec mon stack

En recherchant des vulnérabilités connues touchant PHP et les bibliothèques utilisées dans le projet, j'ai identifié deux cas particulièrement parlants.

CVE-2024-4577 — injection d'arguments PHP-CGI. Cette vulnérabilité critique de PHP, publiée en juin 2024, permet à un attaquant d'exécuter du code arbitraire à distance sur certains serveurs exécutant PHP en mode CGI sous Windows, en exploitant un comportement de conversion de caractères du système. Elle a été massivement exploitée peu après sa divulgation, notamment par des campagnes de rançongiciels. Cette recherche m'a rappelé l'importance de maintenir PHP à jour (des correctifs ont été publiés rapidement pour les versions supportées) et de connaître précisément son propre mode d'exécution : mon environnement de développement (PHP en module Apache sous macOS) et mon environnement de production (FrankenPHP sur Railway) ne correspondent pas au mode d'exécution concerné par cette faille, et j'utilise une version de PHP 8.3 postérieure aux correctifs.

CVE-2016-10033 — exécution de code à distance dans PHPMailer. Cette vulnérabilité historique touchait les versions de PHPMailer antérieures à 5.2.18 : une adresse d'expéditeur spécialement forgée permettait d'injecter des paramètres dans l'appel au programme `send-mail` et d'aboutir à une exécution de code sur le serveur. PHPMailer étant utilisé dans mon projet pour l'envoi des emails transactionnels, ce cas m'a montré qu'une bibliothèque très répandue peut avoir comporté des failles graves par le passé : j'en retiens l'importance d'installer les bibliothèques via Composer (qui permet de vérifier et mettre à jour facilement les versions) plutôt que par copie manuelle de fichiers, et de penser à exécuter `composer audit` pour repérer d'éventuelles dépendances vulnérables.

6.5 Ce que cette veille a changé concrètement dans le projet

Cette recherche m'a conforté dans plusieurs choix déjà présents dans le projet — requêtes préparées, hachage bcrypt, middleware de contrôle d'accès, expiration de session — en me donnant une justification plus précise, appuyée sur des catégories de risques reconnues plutôt que sur une simple intuition.

Le déploiement en production a aussi été l'occasion d'une vigilance concrète sur l'exposition des fichiers sensibles : les fichiers `.env`, `composer.json` et `composer.lock` ont été exclus de

tout accès public, et les identifiants de connexion aux bases de données et au compte email ont été gérés exclusivement via les variables d'environnement de la plateforme d'hébergement (Railway), jamais copiés en dur dans le code source. Cette précaution rejoint la catégorie A05 de l'OWASP Top 10 (« Security Misconfiguration »), qui recouvre notamment l'exposition accidentelle de fichiers de configuration ou de secrets sur un serveur mal paramétré.

6.6 Conclusion

Cette veille, menée à l'occasion de ce projet, m'a permis de relier des choix techniques concrets à des catégories de risques reconnues et à des vulnérabilités réellement documentées, plutôt que de les justifier uniquement par habitude. Je considère cette démarche comme un point de départ : les vulnérabilités évoluent en permanence, et une pratique de veille plus régulière est un objectif que je souhaite développer, notamment dans le cadre de mon alternance.